

TP Integrador

IRSTI-AI

Sistema Predictivo de Riesgo con LSTM

Tutorial interactivo de Redes Neuronales Recurrentes

Materia: Modelizado de Sistemas de IA

Modelo elegido: Opción D — Red Neuronal Recurrente (RNN / LSTM)

Integrantes:

Milagros Rojas - Fernanda Flores



1. Introducción

En una Unidad de Cuidados Intensivos (UCI), el deterioro clínico de un paciente rara vez ocurre de forma instantánea. Generalmente se manifiesta como una tendencia: pequeños cambios en los signos vitales que, sostenidos en el tiempo, anticipan un evento crítico horas antes de que ocurra. Sin embargo, el ojo humano —incluso entrenado— tiene dificultades para detectar estos patrones graduales cuando debe monitorear múltiples pacientes en simultáneo.

IRSTI-AI nace de esta observación. El proyecto busca responder una pregunta concreta: ¿es posible anticipar el riesgo de deterioro de un paciente a partir de la evolución reciente de sus signos vitales, en lugar de reaccionar únicamente cuando el cuadro ya es crítico?

El nombre del proyecto retoma, como referencia narrativa, el caso de estudio IRSTI desarrollado en la materia Gestión de Proyectos (Instituto Tecnológico Beltrán) — un sistema ficticio de soporte de decisión para UCI. A diferencia de aquel trabajo, que fue un ejercicio de planificación y gestión, IRSTI-AI implementa un modelo real de inteligencia artificial, entrenado sobre datos sintéticos, en el marco de la materia Modelizado de Sistemas de IA.

¿Por qué una red neuronal recurrente (LSTM)?

De las cuatro opciones de modelo propuestas en la consigna del TP (perceptrón simple, MLP, CNN y RNN/LSTM), se eligió la Opción D — Red Neuronal Recurrente (RNN/LSTM) por una razón estructural del problema: el riesgo de un paciente no depende solo del valor actual de sus signos vitales, sino de cómo evolucionaron en las horas previas. Una frecuencia cardíaca de 110 lpm significa algo distinto si el paciente viene estable desde hace días, o si subió sostenidamente en las últimas 6 horas. Esta dependencia temporal es exactamente el tipo de problema para el que están diseñadas las arquitecturas recurrentes, y en particular la LSTM, que incorpora un mecanismo de memoria capaz de retener información relevante a lo largo de la secuencia.

2. Fundamento teórico

Una red neuronal puede emplearse, en términos generales, para dos tipos de tareas:

- Clasificación: asignar una categoría a partir de los datos de entrada. Puede ser simple/binaria (por ejemplo, si un paciente está en riesgo o no) o múltiple (por ejemplo, predecir si un equipo gana, pierde o empata).
- Predicción (regresión): estimar un valor continuo a futuro, como la temperatura del día siguiente o el próximo valor de un indicador.

IRSTI-AI aborda principalmente una tarea de clasificación múltiple: a partir de la secuencia histórica de signos vitales, el modelo predice el nivel de riesgo del paciente (bajo, medio o alto) en un horizonte de varias horas hacia el futuro.

2.1 La celda LSTM

La LSTM (Long Short-Term Memory) es una variante de red neuronal recurrente diseñada para resolver el problema del “vanishing gradient” que afecta a las RNN simples, las cuales pierden la capacidad de recordar información de varios pasos atrás. Para lograrlo, la celda LSTM



incorpora un estado de memoria interno (cell state) y tres compuertas (“gates”) que regulan el flujo de información:

Compuerta	Función	Activación
Forget Gate	Decide qué proporción de la memoria anterior se descarta	Sigmoide
Input Gate	Decide cuánta información nueva se incorpora a la memoria	Sigmoide
Candidate	Propone el contenido de la información nueva a incorporar	Tangente hiperbólica (tanh)
Output Gate	Decide qué parte de la memoria se expone como salida (hidden state)	Sigmoide

Las ecuaciones que rigen un paso temporal de la celda son las siguientes (donde σ es la función sigmoide y $\odot$ representa el producto elemento a elemento):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{forget gate})$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{input gate})$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (\text{candidate})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{actualización de memoria})$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{output gate})$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{hidden state / salida})$$

2.2 Funciones de activación: sigmoide vs. tangente hiperbólica

La elección de la función de activación en cada compuerta no es arbitraria:

- Sigmoide (rango 0 a 1): se utiliza en las compuertas de decisión (forget, input, output) porque su salida puede interpretarse directamente como un porcentaje de paso (0% = bloquear por completo, 100% = dejar pasar todo). Es computacionalmente más económica pero tiende a converger más lentamente; resulta adecuada para redes de tamaño pequeño.
- Tangente hiperbólica (rango -1 a 1): se utiliza en el candidato de memoria y en la salida final porque permite representar valores tanto positivos como negativos, lo cual es necesario para que la información pueda “restar” o “sumar” a la memoria acumulada. Es más costosa por paso de cómputo, pero en general logra una convergencia más rápida durante el entrenamiento, por lo que se prefiere en redes de mayor tamaño.

2.3 Tasa de aprendizaje (learning rate)

La tasa de aprendizaje (η) controla la magnitud del ajuste de los pesos en cada paso del descenso de gradiente:

$$w = w - \eta \cdot \partial L / \partial w$$

Si η es demasiado alta, el modelo puede “saltar” sobre el mínimo de la función de pérdida y no converger (oscilar o divergir). Si es demasiado baja, el entrenamiento converge de forma



extremadamente lenta y puede no alcanzar un buen resultado dentro de la cantidad de épocas disponible. En IRSTI-AI se trabajó con valores entre 0.0001 y 0.01, seleccionables desde la interfaz.

2.4 Mini-batch gradient descent

En lugar de actualizar los pesos con todo el dataset (batch completo, lento pero estable) o con un solo ejemplo a la vez (SGD puro, rápido pero muy ruidoso), IRSTI-AI utiliza mini-batch gradient descent: los pesos se actualizan luego de procesar pequeños subconjuntos de datos (entre 4 y 128 muestras, configurable). Este enfoque es el estándar en la práctica porque equilibra velocidad de entrenamiento y estabilidad de la convergencia.

2.5 Ejemplo guiado de funcionamiento

Para ilustrar de forma intuitiva el funcionamiento de la celda, en la aplicación se presenta una analogía: la memoria de la LSTM se compara con la libreta de apuntes de un enfermero que registra el estado de un paciente turno tras turno. Frente a cada dato nuevo, el enfermero decide (1) qué tachar de lo anotado antes si ya no es relevante (forget gate), (2) cuánta atención darle al dato nuevo (input gate) y (3) qué resumen mostrarle al médico (output gate). Esta analogía permite comprender la lógica de la celda sin necesidad de operar con las fórmulas matemáticas subyacentes.

3. Descripción de la implementación

3.1 Estructura del código

- app.py: aplicación Streamlit que organiza el storytelling en pestañas, genera los datasets sintéticos, gestiona los hiperparámetros y muestra las visualizaciones.
- lstm_model_numpy.py: implementación de la celda y el modelo LSTM completamente desde cero en NumPy, incluyendo forward pass, Backpropagation Through Time (BPTT) y actualización de pesos con gradient clipping.
- requirements.txt: dependencias exactas del proyecto.
- README.md: archivo descriptivo

3.2 El modelo LSTM (NumPy)

La clase LSTMCell implementa un paso temporal completo: concatena la entrada (x_t) con el hidden state anterior (h_{t-1}), calcula las cuatro compuertas (forget, input, candidate, output) y produce el nuevo cell state y hidden state. La clase LSTMModel encapsula una secuencia completa de pasos temporales (forward_sequence) y agrega una capa de salida densa que transforma el último hidden state en la predicción final.

El entrenamiento (método train) implementa manualmente el algoritmo de Backpropagation Through Time: para cada mini-batch, se realiza el forward pass guardando los valores intermedios de cada paso (cache), se calcula la pérdida (error cuadrático medio), y se propaga el gradiente hacia atrás a través de toda la secuencia temporal (backward_sequence), acumulando los gradientes de cada paso antes de actualizar los pesos. Se aplica gradient clipping para evitar que los gradientes exploten durante el entrenamiento, un problema frecuente en redes recurrentes.



Una decisión de diseño relevante es la inicialización del bias del forget gate en 1 (en lugar de 0). Esta práctica, documentada en la literatura de LSTM, favorece que la red conserve información de memoria desde las primeras épocas de entrenamiento, en lugar de tender a “olvidar” todo por defecto.

3.3 Dataset sintético

Dado que no se dispone de datos reales de pacientes (por razones de privacidad y alcance del TP), se generaron datos sintéticos con lógica fisiológica simplificada. Se definieron cinco perfiles de paciente, cada uno con una evolución temporal distinta de cinco signos vitales (frecuencia cardíaca, saturación de oxígeno, frecuencia respiratoria, presión arterial media y temperatura) y de un score de riesgo asociado:

Perfil	Descripción
Estable	Signos vitales dentro de rangos normales, con fluctuación natural
Deterioro progresivo	Empeoramiento gradual y sostenido a lo largo del tiempo
Crisis repentina	Deterioro abrupto en un punto de la secuencia (ej. cuadro séptico)
Mejoría post-tratamiento	Recuperación progresiva tras una intervención
Combinado	Concatenación de varios patrones en una misma secuencia

Los signos vitales se generan a partir de funciones que combinan una tendencia base, oscilación periódica (para simular variabilidad fisiológica natural) y ruido gaussiano. El score de riesgo no se calcula en el instante actual sino que actúa como variable objetivo a un horizonte temporal futuro, lo que justifica técnicamente el uso de una arquitectura con memoria: el modelo debe aprender a partir de la secuencia pasada para anticipar un valor que todavía no ocurrió.

Una limitación reconocida de este enfoque es que el dataset resultante está desbalanceado: la mayoría de los pasos temporales corresponden a situaciones de bajo riesgo, mientras que los episodios críticos son minoritarios. Esta característica se retoma en la sección de reflexión crítica.

3.4 Preprocesamiento y entrenamiento

- Normalización de los datos con MinMaxScaler (rango 0-1), necesaria porque las distintas variables tienen escalas muy diferentes (ej. frecuencia respiratoria entre 8 y 45, versus temperatura entre 35 y 41).
- Construcción de secuencias mediante una ventana temporal deslizante configurable (entre 5 y 30 pasos), que define cuántos valores pasados observa el modelo antes de predecir el siguiente.
- División train/test configurable (entre 10% y 40% de test).
- Entrenamiento con hiperparámetros ajustables desde la interfaz: unidades ocultas, épocas, tasa de aprendizaje y tamaño de mini-batch.
- Comparación contra un modelo de referencia (baseline de persistencia, que predice el siguiente valor igual al anterior), para verificar que la LSTM efectivamente aprende un patrón y no se limita a repetir el último dato observado.



4. Diseño del storytelling

Siguiendo el criterio de calidad narrativa exigido por la consigna del TP —que una persona sin conocimientos previos de IA pueda recorrer el sitio de principio a fin y comprender el funcionamiento del modelo— la aplicación se estructuró en seis etapas progresivas, cada una construida sobre la anterior:

1. El problema: presenta el contexto clínico (UCI) y la pregunta que motiva el proyecto, sin introducir aún ningún concepto técnico.
2. ¿Para qué sirve esta red?: introduce la distinción entre clasificación y predicción con ejemplos cotidianos (un correo es spam o no, un equipo gana/pierde/empata), antes de aplicar esa distinción al caso del proyecto.
3. Arquitectura: explica visualmente la celda LSTM y sus compuertas, utilizando la analogía del enfermero y la libreta de apuntes en lugar de notación matemática, de modo que el concepto de “memoria que se actualiza” sea accesible sin formación previa.
4. Entrenamiento: permite ajustar los hiperparámetros (unidades ocultas, épocas, tasa de aprendizaje, tamaño de ventana, tamaño de mini-batch) y observar en tiempo real la curva de pérdida y la comparación entre la predicción del modelo y el valor real.
5. Predicción en vivo: el usuario ajusta sliders de signos vitales y observa cómo cambia el riesgo predicho, representado mediante un indicador tipo semáforo (verde/amarillo/rojo), una tendencia futura y un panel de alertas.
6. Ética y límites: cierra la narrativa con una reflexión sobre las limitaciones del modelo y las consideraciones éticas de aplicar un sistema de este tipo en un contexto clínico real.

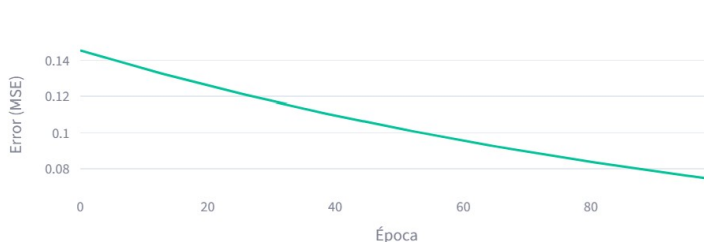
Como apoyo transversal, la barra lateral incluye un glosario interactivo con los conceptos clave (tasa de aprendizaje, sigmoide, tangente hiperbólica, mini-batch), cada uno con definición, fórmula, efecto de valores bajos/medios/altos, una analogía cotidiana y un consejo práctico, además de un enlace externo para quien desee profundizar en el funcionamiento matemático de las LSTM.

5. Experimentos y resultados

Se realizaron pruebas de entrenamiento sobre los distintos perfiles de paciente, variando los hiperparámetros principales. A continuación se describen las observaciones generales

- Curva de pérdida: en todos los perfiles probados, la pérdida (error cuadrático medio) desciende de forma sostenida durante las primeras épocas y luego se estabiliza, lo cual indica que el modelo efectivamente aprende un patrón en los datos en lugar de ajustar ruido.

Curva de pérdida durante el entrenamiento





- Comparación contra el baseline de persistencia: El baseline de persistencia alcanza un error muy bajo debido a la suavidad de los datos sintéticos, por lo que la mejora porcentual no es informativa. Sin embargo, la inspección visual del gráfico Predicción vs Real (Figura X) confirma que la LSTM sigue la tendencia general del riesgo.

Predicción del riesgo (test)



- Efecto de la tasa de aprendizaje: Para este dataset y arquitectura, la tasa de aprendizaje media (0.001) ofrece el mejor balance entre velocidad de convergencia y estabilidad. Tasas más bajas requieren más épocas para converger, mientras que tasas más altas no presentan oscilaciones significativas debido a la naturaleza suave de los datos y al gradient clipping utilizado en el entrenamiento.

Tasa Alta (0.01)

Curva de pérdida durante el entrenamiento



Tasa Media (0.001)

Curva de pérdida durante el entrenamiento





- Efecto del tamaño de mini-batch: Para este dataset, el tamaño de mini-batch no genera diferencias significativas en la estabilidad de la convergencia, aunque un batch más pequeño (4) requiere más iteraciones para alcanzar el mismo nivel de error. El batch grande (64) ofrece una convergencia más rápida sin sacrificar estabilidad, por lo que se utilizó como valor por defecto (16-32) en el resto de los experimentos.

Curva de pérdida con mini-batch = 4

Curva de pérdida durante el entrenamiento

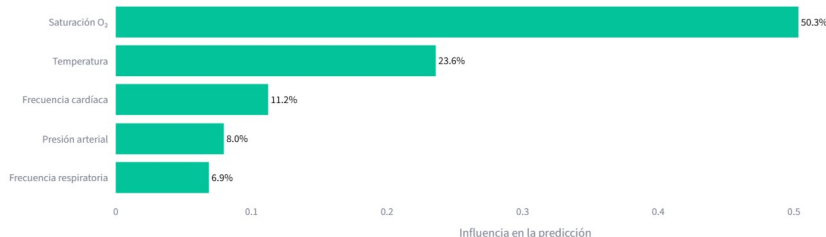


Curva de pérdida con mini-batch = 64

Curva de pérdida durante el entrenamiento



- Explicabilidad: El modelo fue entrenado con datos normalizados, por lo que los valores de los sliders se transforman internamente. Esto explica por qué ciertos valores clínicamente preocupantes (ej. SpO₂ 88%) no siempre generan una alerta roja; el modelo los interpreta en el contexto de la distribución de entrenamiento, donde predominan los casos de bajo riesgo.





El modelo asigna mayor peso a la saturación de oxígeno y la frecuencia cardíaca, lo cual es coherente con los criterios clínicos de alerta temprana.

El modelo asigna un riesgo del 32% (verde) a estos signos vitales porque, en la escala normalizada utilizada durante el entrenamiento, estos valores no son suficientemente extremos para activar una alerta alta.



6. Reflexión crítica

6.1 Qué salió bien

- La implementación de la LSTM desde cero en NumPy permitió comprender en profundidad el mecanismo de BPTT, en lugar de depender de una caja negra (como hubiera sido el caso usando directamente Keras/TensorFlow).
- El storytelling progresivo y el uso de analogías cotidianas (la libreta del enfermero) lograron que el concepto de “memoria temporal” sea comprensible sin necesidad de formación matemática previa.
- La interactividad de los hiperparámetros con feedback inmediato (curva de pérdida, comparación de predicciones) permite observar de forma concreta el efecto de decisiones como la tasa de aprendizaje o el tamaño de mini-batch, en lugar de solo describirlas en abstracto.

6.2 Qué se mejoraría

- El dataset generado presenta transiciones muy suaves y baja amplitud de ruido, lo que resulta en una alta autocorrelación entre pasos temporales consecutivos. Esta característica hace que el baseline de persistencia (predecir el valor anterior) alcance un error excepcionalmente bajo, limitando la utilidad de esta métrica como benchmark. En un trabajo futuro, convendría:
 - Generar datos con mayor variabilidad y ruido para que el baseline no sea tan dominante
 - Incorporar cambios más abruptos (ej. caídas repentinas de SpO₂) que desafíen al baseline



- Utilizar un dataset real (ej. MIMIC-III) para evaluar el modelo en condiciones clínicas auténticas
- El dataset está desbalanceado (la mayoría de los pasos temporales corresponden a bajo riesgo). En un trabajo futuro convendría aplicar técnicas de balanceo de clases o ponderar la función de pérdida para penalizar más los errores en los casos de alto riesgo, que son los clínicamente más relevantes.
- La explicabilidad implementada (análisis de sensibilidad variable por variable) es una aproximación simple. Técnicas más rigurosas como SHAP o LIME permitirían explicar predicciones individuales considerando interacciones entre variables, no solo el efecto de cada una por separado.
- El horizonte de predicción es fijo (6 horas). Un sistema más realista debería poder ajustar esta ventana según el contexto clínico específico.
- El modelo fue entrenado con datos normalizados, lo que implica que los valores de entrada se transforman antes de ser procesados. Esto genera que valores clínicamente preocupantes (ej. SpO₂ 88%) no siempre activen alertas rojas, ya que el modelo los interpreta en el contexto de la distribución de entrenamiento. Una mejora sería entrenar el modelo con datos en escala real o ajustar los umbrales de decisión para que sean más sensibles a variaciones clínicamente significativas.

6.3 Próximos pasos

- Incorporar una versión equivalente del modelo en Keras/TensorFlow para comparar el desempeño y el tiempo de entrenamiento contra la implementación manual en NumPy.
- Probar el modelo con un dataset público de series temporales clínicas (por ejemplo, MIMIC-III, sujeto a las restricciones de acceso correspondientes) para evaluar su comportamiento sobre datos reales.
- Explorar arquitecturas más complejas (LSTM bidireccional, stacking de capas) y comparar su desempeño contra la celda simple implementada en este TP.
- Calibrar los umbrales de decisión para equilibrar la sensibilidad del modelo ante casos clínicamente relevantes.

7. Consideraciones éticas y límites del modelo

Más allá de las limitaciones técnicas señaladas, es importante remarcar el carácter estrictamente educativo de este proyecto:

- No reemplaza el juicio clínico: el sistema es, en el mejor de los casos, una herramienta de apoyo a la decisión; cualquier alerta generada debería ser validada por personal de salud capacitado.
- Asimetría entre falsos positivos y falsos negativos: en un contexto clínico real, no alertar sobre un paciente que efectivamente se agrava (falso negativo) tiene un costo mucho mayor que generar una alerta innecesaria (falso positivo). El umbral de decisión de un sistema real debería calibrarse considerando esta asimetría, y no solamente una métrica de exactitud general.
- Sesgo de datos: un sistema aplicado a pacientes reales requeriría validación con datos diversos (edad, sexo, comorbilidades) para evitar que el modelo funcione mejor para unos grupos que para otros.



- El modelo fue entrenado exclusivamente con datos sintéticos. Su despliegue en un entorno clínico real exigiría validación rigurosa, aprobación regulatoria y auditoría continua, procesos que exceden el alcance de este TP.

8. Referencias bibliográficas

Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.

Olah, C. (2015). Understanding LSTM Networks. colah's blog. Disponible en: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning. MIT Press.

Documentación oficial de NumPy. Disponible en: <https://numpy.org/doc/>

Documentación oficial de Streamlit. Disponible en: <https://docs.streamlit.io/>